

Ejercicio 9

- Expresa la función del tiempo de ejecución de cada uno de los siguientes algoritmos, resuélvala y calcule el orden.
- Compare el tiempo de ejecución del método 'rec2' con el del método 'rec1'.
- Implemente un algoritmo más eficiente que el del método 'rec3'. (es decir, que el $T(n)$ sea menor).

```
static public int rec1(int n){
    if (n <= 1)
        return 1;
    else
        return (rec1(n-1) + rec1(n-1));
}

static public int rec2(int n){
    if (n <= 1)
        return 1;
    else
        return (2 * rec2(n-1));
}

static public int rec3(int n){
    if ( n == 0 )
        return 0;
    else {
        if ( n == 1 )
            return 1;
        else
            return (rec3(n-2) * rec3(n-2));
    }
}
```

a.

rec1

Por un lado tenemos una operación constante para los valores de

$n \leq 1$ que vamos a representar con la constante c_1 .
 Por otro lado, para el resto de valores de n , vemos que se realizan dos llamadas recursivas enviando como parámetro $n - 1$:

$$c_2 + 2 \cdot T(n - 1)$$

Podemos expresar a $T(n)$ como

$$T(n) = \begin{cases} c_1 & n \leq 1, \\ c_2 + 2 \cdot T(n - 1) & n > 1. \end{cases}$$

Analizamos el desarrollo de la llamada recursiva para encontrar el paso general.

1. $T(n) = c_2 + 2 \cdot T(n - 1)$
2. $T(n) = c_2 + 2(c_2 + 2 \cdot T(n - 2)) = 3c_2 + 2 \cdot 2 \cdot T(n - 2)$
3. $T(n) = 3c_2 + 2^2(c_2 + 2 \cdot T(n - 3)) = 7c_2 + 2^2 \cdot 2 \cdot T(n - 3)$
- $\vdots$

Paso i :

$$T(n) = (2^i - 1) \cdot c_2 + 2^i \cdot T(n - i)$$

Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.

Como el caso base se ejecuta cuando $n \leq 1$, vamos a tomar el 1.

O sea,

$$\begin{aligned} n - i &= 1 \\ -i &= 1 - n \\ i &= n - 1 \end{aligned}$$

Reemplazamos el caso base en la función que obtuvimos como paso general.

$$\begin{aligned} T(n) &= (2^{n-1} - 1) \cdot c_2 + 2^{n-1} \cdot T(n - n + 1) \\ &= 2^{n-1} \cdot c_2 - c_2 + 2^{n-1} \cdot T(1) \\ &= 2^n \cdot 2^{-1} \cdot c_2 - 2^n \cdot 2^{-1} \cdot c_1 - c_2 \\ &= 2^n \cdot (2^{-1}c_2 + 2^{-1}c_1) - c_2 \end{aligned}$$

Renombramos

- $c_3 = 2^{-1}c_2 + 2^{-1}c_1$
- $c_4 = -c_2$

$$= 2^n \cdot c_3 + c_4$$

Entonces, la función de tiempo de ejecución de `rec1` es

$$T(n) = 2^n \cdot c_3 + c_4$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$T(n) = 2^n \cdot c_3 + c_4 \leq O(2^n)$$

Hay que buscar los valores constantes c y n_0 tales que

$$2^n \cdot c_3 + c_4 \leq c \cdot 2^n, \text{ para todo } n \geq n_0$$

- Primer término, $2^n \cdot c_3$
Se cumple que $2^n \leq 2^n$. Si multiplicamos a ambos lados por c_3 .

$$2^n c_3 \leq 2^n c_3$$

Con esto hallamos el valor de k_1 , es $k_1 = c_3$. Esta desigualdad vale para $n_0 = 0$.

- Segundo término, c_4 Se cumple que $1 \leq 2^n$.
Si multiplicamos a ambos lados por c_4 .

$$c_4 \leq c_4 \cdot 2^n$$

Con esto hallamos el valor de k_2 , es $k_2 = c_4$. Con $n_0 = 0$ la desigualdad no se cumple, así que vamos a tomar $n_0 = 1$. La desigualdad se cumple para los $n \geq 1$.

Sumamos todos los miembros los resultados anteriores de cada lado.

$$2^n c_3 + c_4 \leq 2^n k_1 + 2^n k_2$$

$$T(n) \leq 2^n (k_1 + k_2)$$

$$T(n) \leq 2^n (c_3 + c_4)$$

$$\text{Renombramos, } c = c_3 + c_4$$

$$T(n) \leq 2^n \cdot c$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(2^n), \text{ con } c = c_3 + c_4 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$

rec2

Por un lado tenemos una operación constante para los valores de $n \leq 1$ que vamos a representar con la constante c_1 .

Por otro lado, para el resto de valores de n , vemos que se realiza una llamada recursiva enviando como parámetro $n - 1$:

$$c_2 + 2 \cdot T(n - 1)$$

Podemos expresar a $T(n)$ como

$$T(n) = \begin{cases} c_1 & n \leq 1, \\ c_2 + T(n - 1) & n > 1. \end{cases}$$

Analizamos el desarrollo de la llamada recursiva para encontrar el paso general.

1. $T(n) = c_2 + T(n - 1)$
2. $T(n) = c_2 + (c_2 + T(n - 2)) = 2 \cdot c_2 + T(n - 2)$
3. $T(n) = 2c_2 + (c_2 + T(n - 3)) = 3c_2 + T(n - 3)$
- $\vdots$

Paso i :

$$T(n) = i \cdot c_2 + T(n - i)$$

Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.

Como el caso base se ejecuta cuando $n \leq 1$, vamos a tomar el 1.

O sea,

$$\begin{aligned} n - i &= 1 \\ -i &= 1 - n \\ i &= n - 1 \end{aligned}$$

Reemplazamos el caso base en la función que obtuvimos como paso general.

$$\begin{aligned} T(n) &= (n - 1) \cdot c_2 + T(n - (n - 1)) \\ &= c_2 \cdot n - c_2 + T(1) \\ &= c_2 \cdot n - c_2 + c_1 \end{aligned}$$

$$\begin{aligned} \text{Renombrando } c_3 &= -c_2 + c_1 \\ &= c_2 n + c_3 \end{aligned}$$

Entonces, la función de tiempo de ejecución de `rec1` es

$$\boxed{T(n) = c_2 n + c_3}$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$\boxed{T(n) = c_2 n + c_3 \leq O(n)}$$

Hay que buscar los valores constantes c y n_0 tales que

$$c_2 n + c_3 \leq c \cdot n, \text{ para todo } n \geq n_0$$

Primer término, $n \cdot c_2$

Se cumple que $n \leq n$. Si multiplicamos a ambos lados por c_2 .

$$nc_2 \leq nc_2$$

Con esto hallamos el valor de k_1 , es $k_1 = c_2$. Esta desigualdad vale para $n_0 = 0$.

Segundo término, c_3

Se cumple que $1 \leq n$.

Si multiplicamos a ambos lados por c_3 .

$$c_3 \leq c_3 \cdot n$$

Con esto hallamos el valor de k_2 , es $k_2 = c_3$. Con $n_0 = 0$ la desigualdad no se cumple, así que vamos a tomar $n_0 = 1$. La desigualdad se cumple para los $n \geq 1$.

Sumamos todos los miembros los resultados anteriores de cada lado.

$$c_2 \cdot n + c_3 \leq k_1 \cdot n + k_2 \cdot n$$

$$T(n) \leq (k_1 + k_2) \cdot n$$

$$T(n) \leq (c_2 + c_3) \cdot n$$

Renombrando $c = c_2 + c_3$

$$T(n) \leq c \cdot n$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n), \text{ con } c = c_2 + c_3 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$

rec3

Por un lado tenemos una operación constante para los valores de $n \leq 1$ que vamos a representar con la constante c_1 .

Por otro lado, para el resto de valores de n , vemos que se realizan dos llamadas recursivas enviando como parámetro $n - 2$:

$$c_2 + 2 \cdot T(n - 2)$$

Podemos expresar a $T(n)$ como

$$T(n) = \begin{cases} c_1 & n \leq 1, \\ c_2 + 2 \cdot T(n - 2) & n > 1. \end{cases}$$

Analizamos el desarrollo de la llamada recursiva para encontrar el paso general.

$$1. T(n) = c_2 + 2 \cdot T(n - 2)$$

$$\begin{aligned}
2. \quad T(n) &= c_2 + 2(c_2 + 2 \cdot T(n-4)) = 3c_2 + 2 \cdot 2 \cdot T(n-4) \\
3. \quad T(n) &= 3c_2 + 2^2(c_2 + 2 \cdot T(n-6)) = 7c_2 + 2^2 \cdot 2 \cdot T(n-6) \\
&\vdots
\end{aligned}$$

Paso i :

$$T(n) = (2^i - 1) \cdot c_2 + 2^i \cdot T(n - 2 \cdot i)$$

Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.

Como el caso base se ejecuta cuando $n \leq 1$, vamos a tomar el 1.

O sea,

$$\begin{aligned}
n - 2 \cdot i &= 1 \\
-2 \cdot i &= 1 - n \\
i &= \frac{n-1}{2}
\end{aligned}$$

Reemplazamos el caso base en la función que obtuvimos como paso general.

$$\begin{aligned}
T(n) &= (2^{\frac{n-1}{2}} - 1) \cdot c_2 + 2^{\frac{n-1}{2}} \cdot T(n - 2(\frac{n-1}{2})) \\
&= (2^{\frac{n}{2}} \cdot 2^{-\frac{1}{2}} - 1) \cdot c_2 + 2^{\frac{n}{2}} \cdot 2^{-\frac{1}{2}} \cdot T(n - (n-1)) \\
&= 2^{\frac{n}{2}} \cdot 2^{-\frac{1}{2}} \cdot c_2 - c_2 + 2^{\frac{n}{2}} \cdot 2^{-\frac{1}{2}} \cdot c_1 \\
&= 2^{\frac{n}{2}} \cdot (2^{-\frac{1}{2}} c_2 + 2^{-\frac{1}{2}} c_1) - c_2
\end{aligned}$$

Renombrando

- $c_3 = 2^{-\frac{1}{2}} c_2 + 2^{-\frac{1}{2}} c_1$
- $c_4 = -c_2$

Entonces, la función de tiempo de ejecución de rec3 es

$$\boxed{T(n) = 2^{\frac{n}{2}} \cdot c_3 + c_4}$$

Otra forma de escribir la función es $T(n) = \sqrt{2^n} \cdot c_3 + c_4$ Vamos a demostrar el orden de la función. Hay que demostrar que

$$\boxed{T(n) = \sqrt{2^n} \cdot c_3 + c_4 \leq O(n)}$$

Hay que buscar los valores constantes c y n_0 tales que

$$\sqrt{2^n} \cdot c_3 + c_4 \leq \sqrt{2^n}, \text{ para todo } n \geq n_0$$

Analizamos término a término.

- Primer término, $\sqrt{2^n} \cdot c_3$
Se cumple que $\sqrt{2^n}$. Si multiplicamos a ambos lados por c_3 .

$$\sqrt{2^n} \cdot c_3 \leq \sqrt{2^n} \cdot c_3$$

Con esto hallamos el valor de k_1 , es $k_1 = c_3$. Esta desigualdad vale para $n_0 = 0$. La desigualdad se cumple para los $n \geq 0$.

- Segundo término, c_4
Se cumple que, $1 \leq \sqrt{2^n}$. Si multiplicamos a ambos lados por c_4 .

$$c_4 \leq \sqrt{2^n} \cdot c_4$$

Con esto hallamos el valor de k_2 , es $k_2 = 1$. Esta desigualdad vale para $n_0 = 1$. La desigualdad se cumple para los $n \geq 1$.

Sumamos todos los miembros los resultados anteriores de cada lado.

$$\sqrt{2^n} \cdot c_3 + c_4 \leq \sqrt{2^n} \cdot k_1 + \sqrt{2^n} \cdot k_2$$

$$T(n) \leq (k_1 + k_2) \cdot \sqrt{2^n}$$

$$T(n) \leq (c_3 + c_4) \cdot \sqrt{2^n}$$

Renombrando $c = c_3 + c_4$

$$T(n) \leq c \cdot \sqrt{2^n}$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(\sqrt{2^n}), \text{ con } c = c_3 + c_4 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$

b. El método `rec2` es más eficiente que `rec1` porque $O(n) < O(2^n)$. Mientras más rápido crezca la función, menos eficiente es el algoritmo.

c.

```
static public int (){                                rec3_mejorado(int n){
    return n % 2;
}
```

Este método es más eficiente porque su orden de ejecución es constante, $O(1)$.